

LAPORAN PRAKTIKUM
PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 13
(NODE.JS: PENGENALAN)



Oleh:

Ahmad Al - Farizi 2311104054

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I

PENDAHULUAN

A. Dasar Teori

Node.js adalah platform runtime berbasis JavaScript yang berjalan di sisi server, dikembangkan oleh Ryan Dahl pada tahun 2009. Node.js memungkinkan pengembang untuk menggunakan JavaScript di luar lingkungan browser, yang sebelumnya hanya digunakan untuk pengembangan front-end. Node.js menggunakan mesin JavaScript V8 milik Google yang diintegrasikan dengan lingkungan runtime server, sehingga memungkinkan eksekusi kode dengan performa tinggi.

Salah satu keunggulan Node.js adalah model event-driven dan non-blocking I/O, yang memungkinkan server menangani banyak permintaan secara bersamaan tanpa harus menunggu satu permintaan selesai terlebih dahulu. Node.js beroperasi dalam satu thread utama, namun dengan dukungan pemrograman asinkron, server dapat menangani ribuan permintaan secara simultan tanpa memerlukan banyak thread seperti pada model multi-threaded tradisional.

Node.js didukung oleh ekosistem paket yang sangat besar yang disebut NPM (Node Package Manager), yang memudahkan pengembang dalam mengelola dependensi dan menggunakan modul yang telah tersedia. Selain itu, Node.js juga mendukung berbagai modul bawaan seperti `http`, `fs`, dan `path`, serta standar modul CommonJS yang menggunakan `require()` untuk mengimpor modul dan `module.exports` untuk mengeksport.

Express.js adalah framework Node.js yang digunakan untuk membangun aplikasi web dan API. Express.js dikenal karena kesederhanaan dan skalabilitasnya, serta mengikuti arsitektur middleware yang memungkinkan pengembang untuk dengan mudah mengatur routing, middleware, dan komponen lainnya. Express.js sangat cocok untuk membangun RESTful API yang mengikuti prinsip REST (Representational State Transfer), yaitu arsitektur yang menggunakan protokol HTTP untuk melakukan operasi CRUD (Create, Read, Update, Delete) pada sumber daya.

Dalam pengembangan RESTful API, komunikasi antara klien dan server dilakukan melalui metode HTTP seperti GET, POST, PUT, dan DELETE. API ini memungkinkan klien (seperti browser atau aplikasi mobile) untuk mengakses dan memanipulasi data yang disimpan dalam database, seperti MySQL.

MySQL adalah sistem manajemen basis data relasional yang digunakan untuk menyimpan dan mengelola data. Dalam praktikum ini, MySQL digunakan sebagai penyimpanan data mahasiswa, dengan tabel yang terdiri dari kolom seperti id, nama, nim, jurusan, dan email.

B. Tujuan

1. Mengimplementasikan pembangunan aplikasi web menggunakan Node.js dengan tepat.
2. Mengimplementasikan pembangunan aplikasi web menggunakan framework Node.js (Express.js) dengan tepat.

BAB II

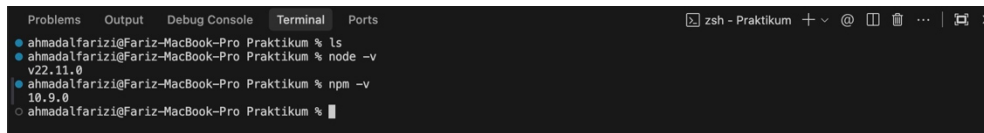
HASIL

A. GUIDED

1. Inisialisasi Project Node.js

a. Persiapan Lingkungan

Sebelum memulai praktikum, perangkat lunak yang harus tersedia adalah Node.js, MySQL (melalui XAMPP), Visual Studio Code, dan Postman. Keberadaan Node.js dan NPM dapat diperiksa melalui perintah:

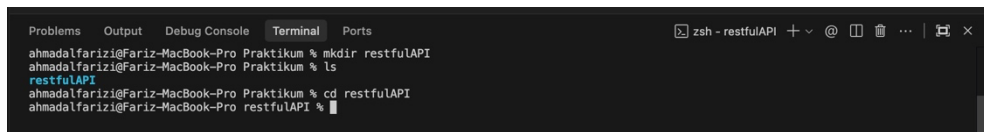


```
Problems Output Debug Console Terminal Ports
ahmadalfarizi@Fariz-MacBook-Pro Praktikum % ls
ahmadalfarizi@Fariz-MacBook-Pro Praktikum % node -v
v22.11.0
ahmadalfarizi@Fariz-MacBook-Pro Praktikum % npm -v
10.9.0
ahmadalfarizi@Fariz-MacBook-Pro Praktikum %
```

Langkah ini bertujuan memastikan bahwa runtime dan package manager telah terpasang dengan benar.

b. Pembuatan Folder Proyek

Folder proyek dibuat dengan nama restfulAPI, yang berfungsi sebagai direktori utama aplikasi Node.js.

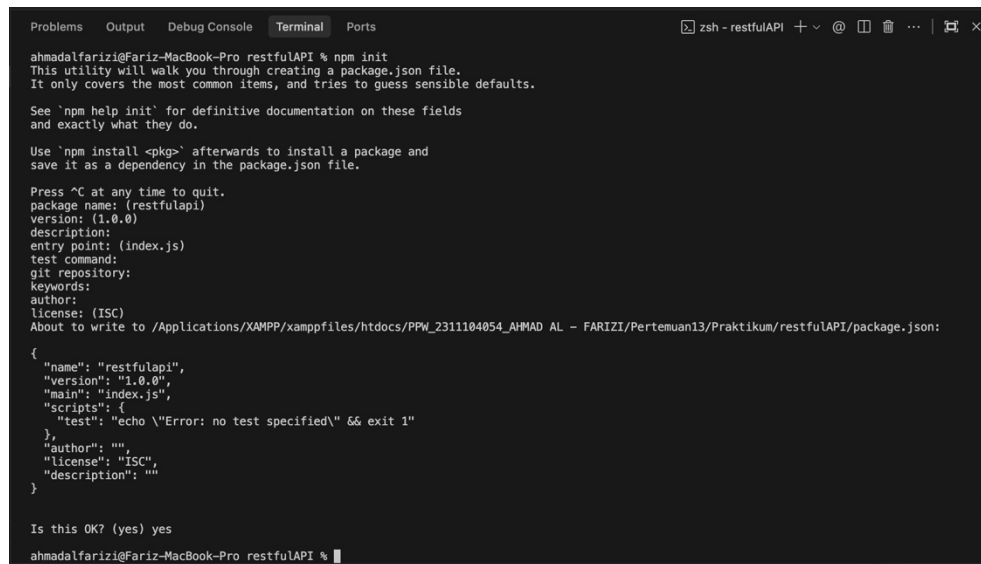


```
Problems Output Debug Console Terminal Ports
ahmadalfarizi@Fariz-MacBook-Pro Praktikum % mkdir restfulAPI
ahmadalfarizi@Fariz-MacBook-Pro Praktikum % ls
restfulAPI
ahmadalfarizi@Fariz-MacBook-Pro Praktikum % cd restfulAPI
ahmadalfarizi@Fariz-MacBook-Pro restfulAPI %
```

Folder ini akan menjadi lokasi penyimpanan seluruh file seperti app.js, db.js, dan crud.js.

c. Instalasi NPM

Proyek Node.js diinisialisasi dengan perintah:



```
ahmadalfarizi@Fariz-MacBook-Pro restfulAPI % npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See 'npm help init' for definitive documentation on these fields
and exactly what they do.

Use 'npm install <pkg>' afterwards to install a package and
save it as a dependency in the package.json file.

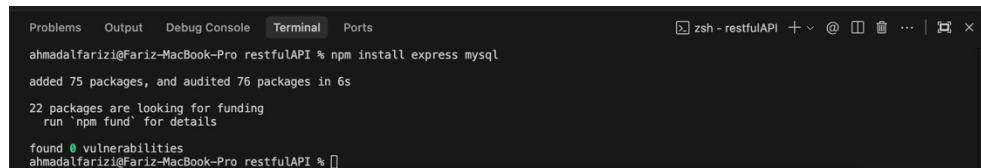
Press ^C at any time to quit.
package name: (restfulapi)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to /Applications/XAMPP/xamppfiles/htdocs/PPW_2311104854_AHMAD AL - FARIZI/Pertemuan13/Praktikum/restfulAPI/package.json:
{
  "name": "restfulapi",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}

Is this OK? (yes) yes
ahmadalfarizi@Fariz-MacBook-Pro restfulAPI %
```

Perintah ini menghasilkan file package.json yang menyimpan informasi proyek dan daftar dependensi yang digunakan.

d. Instalasi Dependency

Framework Express.js dan modul MySQL dipasang dengan perintah:



```
ahmadalfarizi@Fariz-MacBook-Pro restfulAPI % npm install express mysql
added 75 packages, and audited 76 packages in 6s
22 packages are looking for funding
run 'npm fund' for details
found 0 vulnerabilities
ahmadalfarizi@Fariz-MacBook-Pro restfulAPI %
```

Express digunakan untuk membangun RESTful API, sedangkan MySQL digunakan untuk koneksi database.

2. Konfigurasi Database

a. Pembuatan Database dan Tabel

Database akademik dibuat beserta tabel mahasiswa sebagai media penyimpanan data.

```
CREATE DATABASE akademik;
USE akademik;
```

```
CREATE TABLE mahasiswa (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100),
    nim VARCHAR(20),
    jurusan VARCHAR(50),
    email VARCHAR(100)
);
```

Tabel ini digunakan sebagai objek utama pengujian operasi CRUD

#	Nama	Jenis	Penyortiran	Atribut	Tak Ternilai	Bawaan	Komentar	Ekstra	Tindakan
1	id	int(11)			Tidak	Tidak ada		AUTO_INCREMENT	Ubah Hapus Lainnya
2	nama	varchar(100)	utf8mb4_general_ci		Ya	NULL			Ubah Hapus Lainnya
3	nim	varchar(20)	utf8mb4_general_ci		Ya	NULL			Ubah Hapus Lainnya
4	jurusan	varchar(50)	utf8mb4_general_ci		Ya	NULL			Ubah Hapus Lainnya
5	email	varchar(100)	utf8mb4_general_ci		Ya	NULL			Ubah Hapus Lainnya

b. File Koneksi Database (db.js)

File db.js berfungsi mengatur koneksi antara Node.js dan MySQL.

```
const mysql = require('mysql');

const connection = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'akademik'
});

connection.connect(err => {
    if (err) {
        console.error('Koneksi database gagal:', err);
        return;
    }
});
```

```
console.log('Database connected');  
});  
  
module.exports = connection;
```

File ini memungkinkan aplikasi mengakses database secara terpusat.

3. Implementasi RESTful API

a. File CRUD (crud.js)

File crud.js berisi fungsi-fungsi operasi CRUD yang berinteraksi langsung dengan database MySQL.

```
const connection = require("./db");  
  
// Create  
function createMahasiswa(nama, nim, jurusan, email, callback) {  
  const query =  
    "INSERT INTO mahasiswa (nama, nim, jurusan, email) VALUES  
    (?, ?, ?, ?)";  
  connection.query(query, [nama, nim, jurusan, email], (error, results) =>  
  {  
    if (error) {  
      return callback(error, null);  
    }  
    callback(null, results);  
  });  
}  
  
// Read  
function getAllMahasiswa(callback) {  
  const query = "SELECT * FROM mahasiswa";  
  connection.query(query, (error, results) => {  
    if (error) {
```

```

        return callback(error, null);
    }
    callback(null, results);
  });
}

// Update
function updateMahasiswa(id, nama, nim, jurusan, email, callback) {
  const query =
    "UPDATE mahasiswa SET nama = ?, nim = ?, jurusan = ?, email = ?
    WHERE id = ? ";
  connection.query(query, [nama, nim, jurusan, email, id], (error, results)
=> {
    if (error) {
      return callback(error, null);
    }
    if (results.affectedRows === 0) {
      return callback(new Error("No rows updated, ID may not exist"),
null);
    }
    callback(null, results);
  });
}

// Delete
function deleteMahasiswa(id, callback) {
  const query = "DELETE FROM mahasiswa WHERE id = ?";
  connection.query(query, [id], (error, results) => {
    if (error) {
      return callback(error, null);
    }
    callback(null, results);
  });
}

```



```
module.exports = {  
  getAllMahasiswa,  
  createMahasiswa,  
  updateMahasiswa,  
  deleteMahasiswa,  
};
```

Fungsi ini mencakup:

createMahasiswa() untuk menambah data

getAllMahasiswa() untuk membaca data

updateMahasiswa() untuk memperbarui data

deleteMahasiswa() untuk menghapus data

Seluruh fungsi menggunakan query SQL melalui modul MySQL Node.js

b. File Utama Aplikasi (app.js)

File app.js berfungsi sebagai pusat routing aplikasi Express.js. Di dalamnya didefinisikan endpoint RESTful API seperti:

POST /mahasiswaCreate

GET /mahasiswaGet

PUT /mahasiswaUpdate/:id

DELETE /mahasiswaDelete/:id

```
//app.js  
const express = require("express");  
const dbOperations = require("./crud");  
const app = express();  
const port = 3000;  
app.use(express.json());  
// Endpoint untuk menambahkan data  
app.post("/mahasiswaCreate", (req, res) => {  
  const { nama, nim, jurusan, email } = req.body;  
  dbOperations.createMahasiswa(nama, nim, jurusan, email, (error) => {  
    if (error) {
```

```

        return res.status(500).send("Error creating");
    }
    res.status(201).send("Mahasiswa created");
  });
});
// Endpoint untuk mendapatkan semua data
app.get("/mahasiswaGet", (req, res) => {
  dbOperations.getAllMahasiswa((error, users) => {
    if (error) {
      return res.status(500).send("Error fetching users");
    }
    res.json(users);
  });
});
// Endpoint untuk memperbarui data
app.put("/mahasiswaUpdate/:id", (req, res) => {
  const { id } = req.params;
  const { nama, nim, jurusan, email } = req.body;
  dbOperations.updateMahasiswa(id, nama, nim, jurusan, email, (error)
=> {
    if (error) {
      return res.status(500).send("Error updating");
    }
    res.send("Mahasiswa updated");
  });
});
// Endpoint untuk menghapus data
app.delete("/mahasiswaDelete/:id", (req, res) => {
  const { id } = req.params;
  dbOperations.deleteMahasiswa(id, (error) => {
    if (error) {
      return res.status(500).send("Error deleting");
    }
  });
});

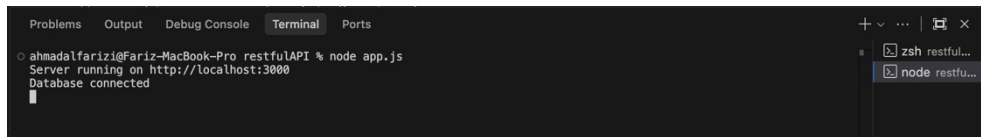
```

```

    }
    res.send("Mahasiswa deleted");
  });
});
// Jalankan server
app.listen(port, () => {
  console.log(`Server running on http://localhost:${port}`);
});

```

Server dijalankan pada port 3000 dengan perintah:



```

ahmadalfarizi@Fariz-MacBook-Pro: ~/restfulAPI % node app.js
Server running on http://localhost:3000
Database connected

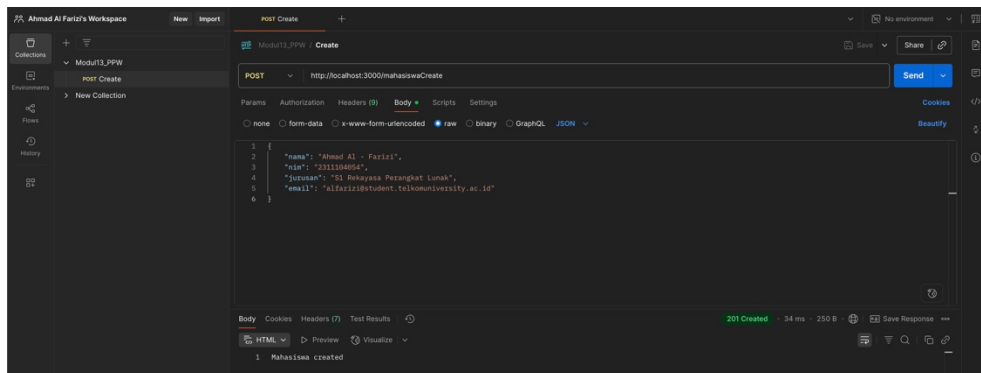
```

Setelah server aktif, API siap menerima request dari klien.

c. Pengujian Menggunakan Postman

Pengujian dilakukan dengan mengirim request POST, GET, PUT, dan DELETE melalui Postman.

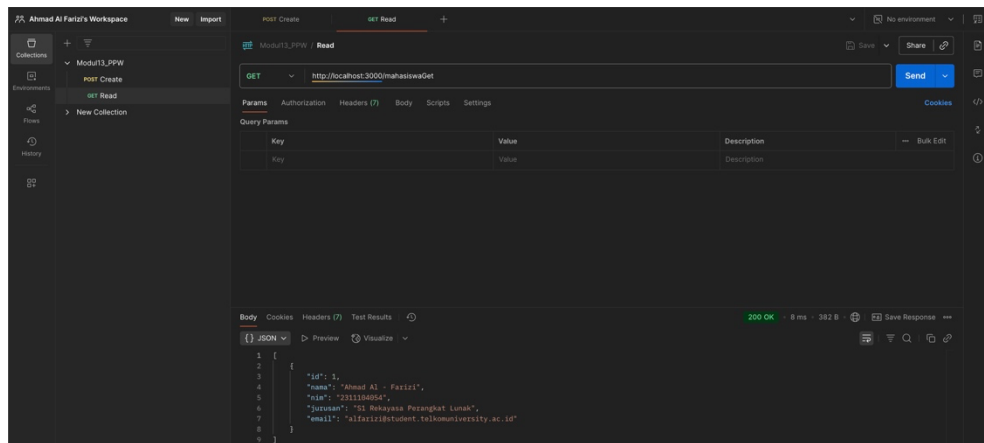
Metode POST digunakan untuk menambahkan data.



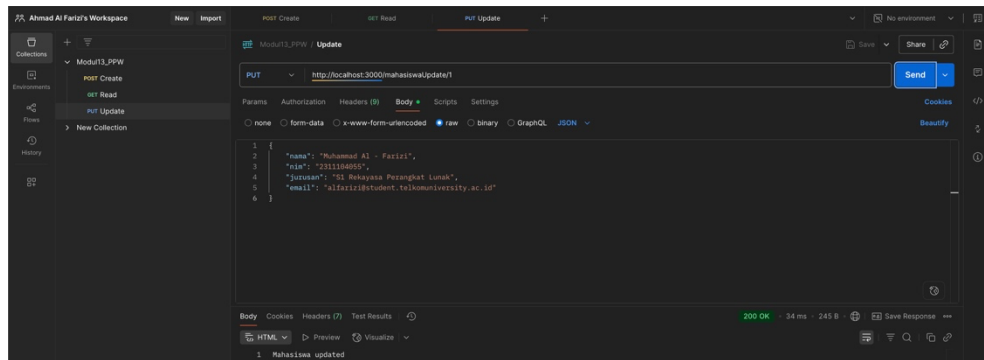
Yang tadinya kosong sekarang sudah ada data yang dibuat.



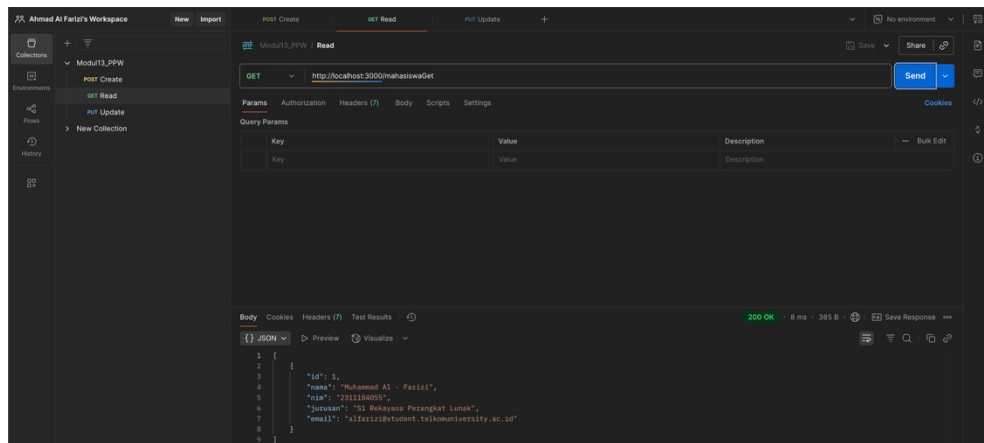
Metode GET untuk membaca data.



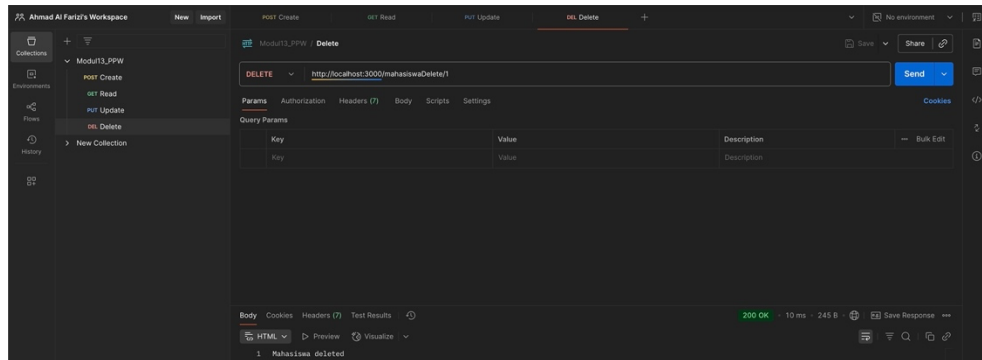
PUT untuk memperbarui.



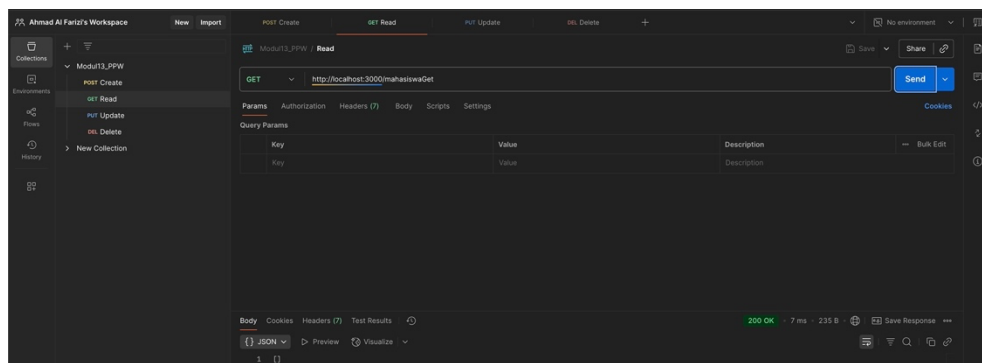
Cek pada Get apakah data sudah berubah.



DELETE untuk menghapus data mahasiswa dari database.



Cek pada Get apakah sudah terhapus.



B. UNGUIDED

Soal:

1. Mahasiswa diminta untuk membangun sebuah aplikasi web sederhana untuk pengelolaan data mahasiswa berbasis Node.js, Express.js, dan MySQL. Aplikasi harus menyediakan RESTful API serta dapat diakses melalui browser menggunakan halaman web sederhana (HTML dan JavaScript). Mahasiswa diperbolehkan menggunakan proyek yang telah dibuat di atas atau membuat proyek baru.

Catatan: Sertakan penjelasan tahapan pengerjaan aplikasi secara sistematis.

Jawaban:

1. Pengelolaan Data Mahasiswa

a. Backend

package.json

```
{
  "name": "latihan-mahasiswa-api",
  "version": "1.0.0",
  "description": "Aplikasi Web Pengelolaan Data Mahasiswa dengan RESTful API",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "author": "Ahmad Al-Farizi",
  "license": "ISC",
  "dependencies": {
    "express": "^5.2.1",
    "mysql": "^2.18.1"
  }
}
```

db.js

```
const mysql = require('mysql');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'akademik'
});

connection.connect(err => {
  if (err) {
```

```

        console.error('❌ Koneksi database gagal:', err.message);
        return;
    }
    console.log('✅ Database connected successfully!');
});

module.exports = connection;

```

crud.js

```

const connection = require('./db');

/**
 * @param {string} nama
 * @param {string} nim
 * @param {string} jurusan
 * @param {string} email
 * @param {function} callback
 */
function createMahasiswa(nama, nim, jurusan, email, callback) {
    const query = 'INSERT INTO mahasiswa (nama, nim, jurusan, email) VALUES (?, ?, ?, ?)';
    connection.query(query, [nama, nim, jurusan, email], (error, results) => {
        if (error) {
            return callback(error, null);
        }
        callback(null, results);
    });
}

/**
 * @param {function} callback
 */

```

```

*/

function getAllMahasiswa(callback) {
    const query = 'SELECT * FROM mahasiswa ORDER BY id
DESC';
    connection.query(query, (error, results) => {
        if (error) {
            return callback(error, null);
        }
        callback(null, results);
    });
}

/**
 * @param {number} id
 * @param {function} callback
 */

function getMahasiswaById(id, callback) {
    const query = 'SELECT * FROM mahasiswa WHERE id = ?';
    connection.query(query, [id], (error, results) => {
        if (error) {
            return callback(error, null);
        }
        callback(null, results[0] || null);
    });
}

/**
 * @param {number} id
 * @param {string} nama
 * @param {string} nim
 * @param {string} jurusan
 * @param {string} email

```



```

* @param {function} callback - Callback function (error, results)
*/

function updateMahasiswa(id, nama, nim, jurusan, email,
callback) {
    const query = 'UPDATE mahasiswa SET nama = ?, nim = ?,
jurusan = ?, email = ? WHERE id = ?';
    connection.query(query, [nama, nim, jurusan, email, id], (error,
results) => {
        if (error) {
            return callback(error, null);
        }
        if (results.affectedRows === 0) {
            return callback(new Error('Data tidak ditemukan'), null);
        }
        callback(null, results);
    });
}

/**
* @param {number} id
* @param {function} callback
*/

function deleteMahasiswa(id, callback) {
    const query = 'DELETE FROM mahasiswa WHERE id = ?';
    connection.query(query, [id], (error, results) => {
        if (error) {
            return callback(error, null);
        }
        if (results.affectedRows === 0) {
            return callback(new Error('Data tidak ditemukan'), null);
        }
        callback(null, results);
    });
}

```

```
});  
}  
  
module.exports = {  
  createMahasiswa,  
  getAllMahasiswa,  
  getMahasiswaById,  
  updateMahasiswa,  
  deleteMahasiswa  
};
```

app.js

```
const express = require('express');  
const path = require('path');  
const dbOperations = require('./crud');  
  
const app = express();  
const PORT = 3000;  
  
app.use(express.json());  
  
app.use(express.static(path.join(__dirname, 'public')));  
  
app.get('/api/mahasiswa', (req, res) => {  
  dbOperations.getAllMahasiswa((error, data) => {  
    if (error) {  
      return res.status(500).json({  
        success: false,  
        message: 'Gagal mengambil data mahasiswa',  
        error: error.message  
      });  
    }  
  })  
});
```

```
        res.json({
            success: true,
            message: 'Data berhasil diambil',
            data: data
        });
    });
});

app.get('/api/mahasiswa/:id', (req, res) => {
    const { id } = req.params;
    dbOperations.getMahasiswaById(id, (error, data) => {
        if (error) {
            return res.status(500).json({
                success: false,
                message: 'Gagal mengambil data mahasiswa',
                error: error.message
            });
        }
        if (!data) {
            return res.status(404).json({
                success: false,
                message: 'Data mahasiswa tidak ditemukan'
            });
        }
        res.json({
            success: true,
            message: 'Data berhasil diambil',
            data: data
        });
    });
});
```

```
app.post('/api/mahasiswa', (req, res) => {
  const { nama, nim, jurusan, email } = req.body;

  if (!nama || !nim || !jurusan || !email) {
    return res.status(400).json({
      success: false,
      message: 'Semua field harus diisi (nama, nim, jurusan, email)'
    });
  }

  dbOperations.createMahasiswa(nama, nim, jurusan, email,
(error, result) => {
    if (error) {
      return res.status(500).json({
        success: false,
        message: 'Gagal menambahkan data mahasiswa',
        error: error.message
      });
    }
    res.status(201).json({
      success: true,
      message: 'Data mahasiswa berhasil ditambahkan',
      data: { id: result.insertId, nama, nim, jurusan, email }
    });
  });
});

app.put('/api/mahasiswa/:id', (req, res) => {
  const { id } = req.params;
  const { nama, nim, jurusan, email } = req.body;
```

```

    if (!nama || !nim || !jurusan || !email) {
      return res.status(400).json({
        success: false,
        message: 'Semua field harus diisi (nama, nim, jurusan,
email)'
      });
    }

    dbOperations.updateMahasiswa(id, nama, nim, jurusan, email,
(error, result) => {
      if (error) {
        return res.status(500).json({
          success: false,
          message: 'Gagal memperbarui data mahasiswa',
          error: error.message
        });
      }
      res.json({
        success: true,
        message: 'Data mahasiswa berhasil diperbarui',
        data: { id: parseInt(id), nama, nim, jurusan, email }
      });
    });
  });

app.delete('/api/mahasiswa/:id', (req, res) => {
  const { id } = req.params;
  dbOperations.deleteMahasiswa(id, (error, result) => {
    if (error) {
      return res.status(500).json({
        success: false,
        message: 'Gagal menghapus data mahasiswa',

```

```

        error: error.message
    });
}
res.json({
    success: true,
    message: 'Data mahasiswa berhasil dihapus'
});
});
});

app.listen(PORT, () => {

    console.log('=====
=====');
    console.log(' Aplikasi Pengelolaan Data Mahasiswa');

    console.log('=====
=====');
    console.log('🚀 Server running on http://localhost:${PORT}');
    console.log('📄 Open browser: http://localhost:${PORT}');

    console.log('=====
=====');

});

```

b. Frontend

public/index.html

```

<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">

```

```
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<meta name="description"
  content="Aplikasi Web Pengelolaan Data Mahasiswa -
RESTful API dengan Node.js, Express.js, dan MySQL">
<title>Pengelolaan Data Mahasiswa | Ahmad Al-Farizi</title>
<link rel="stylesheet" href="style.css">
<link
  href="https://fonts.googleapis.com/css2?family=Inter:wght@300;
400;500;600;700&display=swap" rel="stylesheet">
</head>

<body>
  <div class="container">
    <header class="header">
      <div class="header-content">
        <div class="logo">
          <span class="logo-icon">🎓</span>
          <div class="logo-text">
            <h1>Pengelolaan Data Mahasiswa</h1>
            <p>RESTful API dengan Node.js, Express.js &
MySQL</p>
          </div>
        </div>
        <div class="header-info">
          <span class="badge">Latihan Pertemuan 13</span>
        </div>
      </div>
    </header>

    <main class="main-content">
      <section class="form-section">
```

```

<div class="card form-card">
  <div class="card-header">
    <h2 id="formTitle">
      <span class="icon">✚</span>
      Tambah Mahasiswa Baru
    </h2>
  </div>
  <div class="card-body">
    <form id="mahasiswaForm">
      <input type="hidden" id="mahasiswaId">

      <div class="form-group">
        <label for="nama">Nama Lengkap</label>
        <input type="text" id="nama" name="nama"
placeholder="Masukkan nama lengkap" required>
      </div>

      <div class="form-group">
        <label for="nim">NIM</label>
        <input type="text" id="nim" name="nim"
placeholder="Contoh: 2311104054" required>
      </div>

      <div class="form-group">
        <label for="jurusan">Jurusan / Program
Studi</label>
        <input type="text" id="jurusan"
name="jurusan" placeholder="Contoh: Teknik Informatika"
required>
      </div>

      <div class="form-group">

```



```
<label for="email">Email</label>
<input type="email" id="email"
name="email" placeholder="Contoh: mahasiswa@email.com"
required>
</div>
```

```
<div class="form-actions">
  <button type="submit" class="btn btn-
primary" id="submitBtn">
    <span class="btn-icon"><img alt="save icon" data-bbox="708 323 728 343"/></span>
    Simpan Data
  </button>
  <button type="button" class="btn btn-
secondary" id="cancelBtn" style="display: none;">
    <span class="btn-icon"><img alt="cancel icon" data-bbox="708 448 728 468"/></span>
    Batal
  </button>
</div>
</form>
</div>
</div>
</section>
```

```
<section class="table-section">
  <div class="card table-card">
    <div class="card-header">
      <h2>
        <span class="icon"><img alt="table icon" data-bbox="643 763 663 783"/></span>
        Daftar Mahasiswa
      </h2>
      <button class="btn btn-refresh" id="refreshBtn">
        <span class="btn-icon"><img alt="refresh icon" data-bbox="673 863 693 883"/></span>
```

```

Refresh
</button>
</div>
<div class="card-body">
  <div class="table-wrapper">
    <table id="mahasiswaTable">
      <thead>
        <tr>
          <th>No</th>
          <th>Nama</th>
          <th>NIM</th>
          <th>Jurusan</th>
          <th>Email</th>
          <th>Aksi</th>
        </tr>
      </thead>
      <tbody id="tableBody">
      </tbody>
    </table>
  </div>
  <div class="empty-state" id="emptyState"
style="display: none;">
    <span class="empty-icon"><img alt="Empty state icon" data-bbox="695 638 715 650"/></span>
    <p>Belum ada data mahasiswa</p>
    <span class="empty-hint">Tambahkan data
mahasiswa baru menggunakan form di atas</span>
  </div>
  <div class="loading-state" id="loadingState"
style="display: none;">
    <div class="spinner"></div>
    <p>Memuat data...</p>
  </div>

```

```
        </div>
    </div>
</section>
</main>

<footer class="footer">
    <p>© 2026 Ahmad Al-Farizi | NIM: 2311104054 | Telkom
University Purwokerto</p>
</footer>
</div>

<div class="toast-container" id="toastContainer"></div>

<div class="modal-overlay" id="modalOverlay">
    <div class="modal">
        <div class="modal-header">
            <h3 id="modalTitle">Konfirmasi</h3>
        </div>
        <div class="modal-body">
            <p id="modalMessage">Apakah Anda yakin ingin
menghapus data ini?</p>
        </div>
        <div class="modal-footer">
            <button class="btn btn-secondary"
id="modalCancel">Batal</button>
            <button class="btn btn-danger"
id="modalConfirm">Hapus</button>
        </div>
    </div>
</div>

<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

public.style.css

```
:root {  
  --primary-hue: 230;  
  --primary: hsl(var(--primary-hue), 70%, 55%);  
  --primary-light: hsl(var(--primary-hue), 70%, 65%);  
  --primary-dark: hsl(var(--primary-hue), 70%, 45%);  
  --accent: hsl(160, 70%, 45%);  
  --accent-light: hsl(160, 70%, 55%);  
  --danger: hsl(0, 70%, 55%);  
  --danger-light: hsl(0, 70%, 65%);  
  --warning: hsl(45, 90%, 50%);  
  --bg-gradient-start: hsl(var(--primary-hue), 30%, 15%);  
  --bg-gradient-end: hsl(var(--primary-hue), 40%, 8%);  
  --surface: hsla(var(--primary-hue), 20%, 20%, 0.6);  
  --surface-light: hsla(var(--primary-hue), 20%, 25%, 0.8);  
  --border: hsla(var(--primary-hue), 20%, 40%, 0.3);  
  --text-primary: hsl(0, 0%, 95%);  
  --text-secondary: hsl(0, 0%, 70%);  
  --text-muted: hsl(0, 0%, 55%);  
  --space-xs: 0.25rem;  
  --space-sm: 0.5rem;  
  --space-md: 1rem;  
  --space-lg: 1.5rem;  
  --space-xl: 2rem;  
  --space-2xl: 3rem;  
  --radius-sm: 0.5rem;  
  --radius-md: 0.75rem;  
  --radius-lg: 1rem;
```

```
--radius-xl: 1.5rem;

--shadow-sm: 0 2px 8px hsla(0, 0%, 0%, 0.2);
--shadow-md: 0 4px 16px hsla(0, 0%, 0%, 0.25);
--shadow-lg: 0 8px 32px hsla(0, 0%, 0%, 0.3);

--transition-fast: 150ms ease;
--transition-normal: 250ms ease;
--transition-slow: 400ms ease;
}

*,
*::before,
*::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html {
  font-size: 16px;
  scroll-behavior: smooth;
}

body {
  font-family: 'Inter', -apple-system, BlinkMacSystemFont,
'Segoe UI', sans-serif;
  background: linear-gradient(135deg, var(--bg-gradient-start),
var(--bg-gradient-end));
  color: var(--text-primary);
  min-height: 100vh;
  line-height: 1.6;
}
```

```
.container {  
  max-width: 1200px;  
  margin: 0 auto;  
  padding: var(--space-lg);  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
}  
  
.main-content {  
  flex: 1;  
  display: grid;  
  grid-template-columns: 1fr 2fr;  
  gap: var(--space-xl);  
  margin-top: var(--space-xl);  
}  
  
@media (max-width: 968px) {  
  .main-content {  
    grid-template-columns: 1fr;  
  }  
}  
  
.header {  
  background: var(--surface);  
  backdrop-filter: blur(20px);  
  border: 1px solid var(--border);  
  border-radius: var(--radius-xl);  
  padding: var(--space-lg) var(--space-xl);  
  box-shadow: var(--shadow-md);  
}
```

```
.header-content {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  flex-wrap: wrap;  
  gap: var(--space-md);  
}  
  
.logo {  
  display: flex;  
  align-items: center;  
  gap: var(--space-md);  
}  
  
.logo-icon {  
  font-size: 2.5rem;  
  filter: drop-shadow(0 2px 4px hsla(0, 0%, 0%, 0.3));  
}  
  
.logo-text h1 {  
  font-size: 1.5rem;  
  font-weight: 700;  
  background: linear-gradient(135deg, var(--text-primary), var(--  
primary-light));  
  -webkit-background-clip: text;  
  -webkit-text-fill-color: transparent;  
  background-clip: text;  
}  
  
.logo-text p {  
  font-size: 0.875rem;  
  color: var(--text-secondary);
```

```
}

.badge {
  background: linear-gradient(135deg, var(--primary), var(--primary-dark));
  color: white;
  padding: var(--space-sm) var(--space-md);
  border-radius: var(--radius-md);
  font-size: 0.75rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.card {
  background: var(--surface);
  backdrop-filter: blur(20px);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  overflow: hidden;
  box-shadow: var(--shadow-md);
  transition: transform var(--transition-normal), box-shadow var(--transition-normal);
}

.card:hover {
  box-shadow: var(--shadow-lg);
}

.card-header {
  background: var(--surface-light);
  padding: var(--space-md) var(--space-lg);
```



```
border-bottom: 1px solid var(--border);  
display: flex;  
justify-content: space-between;  
align-items: center;  
}
```

```
.card-header h2 {  
  font-size: 1.125rem;  
  font-weight: 600;  
  display: flex;  
  align-items: center;  
  gap: var(--space-sm);  
}
```

```
.card-header .icon {  
  font-size: 1.25rem;  
}
```

```
.card-body {  
  padding: var(--space-lg);  
}
```

```
.form-group {  
  margin-bottom: var(--space-lg);  
}
```

```
.form-group label {  
  display: block;  
  font-size: 0.875rem;  
  font-weight: 500;  
  color: var(--text-secondary);  
  margin-bottom: var(--space-sm);  
}
```

```
}

.form-group input {
  width: 100%;
  padding: var(--space-md);
  background: hsla(var(--primary-hue), 20%, 15%, 0.8);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  color: var(--text-primary);
  font-size: 1rem;
  font-family: inherit;
  transition: border-color var(--transition-fast), box-shadow var(--transition-fast);
}

.form-group input::placeholder {
  color: var(--text-muted);
}

.form-group input:focus {
  outline: none;
  border-color: var(--primary);
  box-shadow: 0 0 0 3px hsla(var(--primary-hue), 70%, 55%, 0.2);
}

.form-actions {
  display: flex;
  gap: var(--space-md);
  margin-top: var(--space-xl);
}
```

```
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: var(--space-sm);
  padding: var(--space-md) var(--space-lg);
  border: none;
  border-radius: var(--radius-md);
  font-size: 0.875rem;
  font-weight: 600;
  font-family: inherit;
  cursor: pointer;
  transition: all var(--transition-fast);
}

.btn-icon {
  font-size: 1rem;
}

.btn-primary {
  background: linear-gradient(135deg, var(--primary), var(--primary-dark));
  color: white;
  box-shadow: 0 4px 12px hsla(var(--primary-hue), 70%, 40%, 0.3);
}

.btn-primary:hover {
  background: linear-gradient(135deg, var(--primary-light), var(--primary));
  transform: translateY(-2px);
}
```

```
    box-shadow: 0 6px 16px hsla(var(--primary-hue), 70%, 40%,
0.4);
}
```

```
.btn-secondary {
    background: var(--surface-light);
    color: var(--text-primary);
    border: 1px solid var(--border);
}
```

```
.btn-secondary:hover {
    background: hsla(var(--primary-hue), 20%, 30%, 0.8);
}
```

```
.btn-danger {
    background: linear-gradient(135deg, var(--danger), hsl(0, 70%,
45%));
    color: white;
}
```

```
.btn-danger:hover {
    background: linear-gradient(135deg, var(--danger-light), var(--
danger));
    transform: translateY(-2px);
}
```

```
.btn-success {
    background: linear-gradient(135deg, var(--accent), hsl(160,
70%, 35%));
    color: white;
}
```

```
.btn-success:hover {  
    background: linear-gradient(135deg, var(--accent-light), var(--  
accent));  
    transform: translateY(-2px);  
}
```

```
.btn-refresh {  
    background: transparent;  
    color: var(--text-secondary);  
    padding: var(--space-sm) var(--space-md);  
}
```

```
.btn-refresh:hover {  
    color: var(--text-primary);  
    background: var(--surface-light);  
}
```

```
.btn-sm {  
    padding: var(--space-sm) var(--space-md);  
    font-size: 0.75rem;  
}
```

```
.table-wrapper {  
    overflow-x: auto;  
}
```

```
table {  
    width: 100%;  
    border-collapse: collapse;  
}
```

```
th,
```

```
td {
  padding: var(--space-md);
  text-align: left;
  border-bottom: 1px solid var(--border);
}

th {
  background: var(--surface-light);
  font-weight: 600;
  font-size: 0.75rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: var(--text-secondary);
}

td {
  font-size: 0.875rem;
}

tr:hover td {
  background: hsla(var(--primary-hue), 20%, 25%, 0.3);
}

.action-buttons {
  display: flex;
  gap: var(--space-sm);
}

.empty-state,
.loading-state {
  text-align: center;
  padding: var(--space-2xl);
}
```

```
        color: var(--text-muted);
    }

.empty-icon {
    font-size: 3rem;
    display: block;
    margin-bottom: var(--space-md);
}

.empty-state p {
    font-size: 1.125rem;
    margin-bottom: var(--space-sm);
    color: var(--text-secondary);
}

.empty-hint {
    font-size: 0.875rem;
}

.spinner {
    width: 40px;
    height: 40px;
    border: 3px solid var(--border);
    border-top-color: var(--primary);
    border-radius: 50%;
    animation: spin 1s linear infinite;
    margin: 0 auto var(--space-md);
}

@keyframes spin {
    to {
        transform: rotate(360deg);
    }
}
```

```
}  
}  
  
.toast-container {  
  position: fixed;  
  bottom: var(--space-lg);  
  right: var(--space-lg);  
  z-index: 1000;  
  display: flex;  
  flex-direction: column;  
  gap: var(--space-sm);  
}  
  
.toast {  
  background: var(--surface);  
  backdrop-filter: blur(20px);  
  border: 1px solid var(--border);  
  border-radius: var(--radius-md);  
  padding: var(--space-md) var(--space-lg);  
  box-shadow: var(--shadow-lg);  
  display: flex;  
  align-items: center;  
  gap: var(--space-md);  
  min-width: 300px;  
  animation: slideIn 300ms ease;  
}  
  
.toast.success {  
  border-left: 4px solid var(--accent);  
}  
  
.toast.error {
```



```
border-left: 4px solid var(--danger);  
}
```

```
.toast.warning {  
  border-left: 4px solid var(--warning);  
}
```

```
.toast-icon {  
  font-size: 1.25rem;  
}
```

```
.toast-message {  
  flex: 1;  
  font-size: 0.875rem;  
}
```

```
.toast-close {  
  background: none;  
  border: none;  
  color: var(--text-muted);  
  cursor: pointer;  
  font-size: 1.25rem;  
  padding: var(--space-xs);  
}
```

```
.toast-close:hover {  
  color: var(--text-primary);  
}
```

```
@keyframes slideIn {  
  from {  
    transform: translateX(100%);
```

```
        opacity: 0;
    }

    to {
        transform: translateX(0);
        opacity: 1;
    }
}

@keyframes slideOut {
    from {
        transform: translateX(0);
        opacity: 1;
    }

    to {
        transform: translateX(100%);
        opacity: 0;
    }
}

.modal-overlay {
    position: fixed;
    inset: 0;
    background: hsla(0, 0%, 0%, 0.7);
    backdrop-filter: blur(4px);
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 1001;
    opacity: 0;
    visibility: hidden;
```

```
        transition: all var(--transition-normal);
    }

    .modal-overlay.active {
        opacity: 1;
        visibility: visible;
    }

    .modal {
        background: var(--surface);
        backdrop-filter: blur(20px);
        border: 1px solid var(--border);
        border-radius: var(--radius-lg);
        width: 90%;
        max-width: 400px;
        transform: scale(0.9);
        transition: transform var(--transition-normal);
    }

    .modal-overlay.active .modal {
        transform: scale(1);
    }

    .modal-header {
        padding: var(--space-lg);
        border-bottom: 1px solid var(--border);
    }

    .modal-header h3 {
        font-size: 1.125rem;
        font-weight: 600;
    }
```

```
.modal-body {  
  padding: var(--space-lg);  
  color: var(--text-secondary);  
}  
  
.modal-footer {  
  padding: var(--space-lg);  
  border-top: 1px solid var(--border);  
  display: flex;  
  justify-content: flex-end;  
  gap: var(--space-md);  
}  
  
.footer {  
  margin-top: var(--space-2xl);  
  text-align: center;  
  padding: var(--space-lg);  
  color: var(--text-muted);  
  font-size: 0.875rem;  
}  
  
@media (max-width: 640px) {  
  .container {  
    padding: var(--space-md);  
  }  
  
  .header-content {  
    flex-direction: column;  
    text-align: center;  
  }  
}
```

```
.logo {  
  flex-direction: column;  
}  
  
.logo-text {  
  text-align: center;  
}  
  
.card-body {  
  padding: var(--space-md);  
}  
  
.form-actions {  
  flex-direction: column;  
}  
  
.btn {  
  width: 100%;  
}  
  
th,  
td {  
  padding: var(--space-sm);  
  font-size: 0.75rem;  
}  
  
.action-buttons {  
  flex-direction: column;  
}  
}
```

public/script.js

```
const API_URL = '/api/mahasiswa';

const elements = {
  form: document.getElementById('mahasiswaForm'),
  formTitle: document.getElementById('formTitle'),
  submitBtn: document.getElementById('submitBtn'),
  cancelBtn: document.getElementById('cancelBtn'),
  refreshBtn: document.getElementById('refreshBtn'),
  tableBody: document.getElementById('tableBody'),
  emptyState: document.getElementById('emptyState'),
  loadingState: document.getElementById('loadingState'),
  toastContainer: document.getElementById('toastContainer'),
  modalOverlay: document.getElementById('modalOverlay'),
  modalTitle: document.getElementById('modalTitle'),
  modalMessage: document.getElementById('modalMessage'),
  modalConfirm: document.getElementById('modalConfirm'),
  modalCancel: document.getElementById('modalCancel'),
  mahasiswaId: document.getElementById('mahasiswaId'),
  nama: document.getElementById('nama'),
  nim: document.getElementById('nim'),
  jurusan: document.getElementById('jurusan'),
  email: document.getElementById('email')
};

let isEditMode = false;
let deleteId = null;

/**
 * @param {string} message
 * @param {string} type
 */
function showToast(message, type = 'success') {
```

```

const icons = {
  success: '✅',
  error: '❌',
  warning: '⚠️'
};

const toast = document.createElement('div');
toast.className = `toast ${type}`;
toast.innerHTML = `
  <span class="toast-icon">${icons[type]}</span>
  <span class="toast-message">${message}</span>
  <button class="toast-close"
onclick="this.parentElement.remove()">×</button>
`;

elements.toastContainer.appendChild(toast);

setTimeout(() => {
  toast.style.animation = 'slideOut 300ms ease forwards';
  setTimeout(() => toast.remove(), 300);
}, 3000);
}

/**
 * @param {boolean} show
 */
function toggleLoading(show) {
  elements.loadingState.style.display = show ? 'block' : 'none';
  elements.tableBody.parentElement.style.display = show ?
'none' : 'table';
}

```

```

/**
 * @param {boolean} show
 */
function toggleEmptyState(show) {
    elements.emptyState.style.display = show ? 'block' : 'none';
    elements.tableBody.parentElement.style.display = show ?
'none' : 'table';
}

/**
 * @param {Object} data
 */
function resetForm() {
    elements.form.reset();
    elements.mahasiswaId.value = "";
    isEditMode = false;
    elements.formTitle.innerHTML = '<span
class="icon">✚</span> Tambah Mahasiswa Baru';
    elements.submitBtn.innerHTML = '<span class="btn-
icon">💾</span> Simpan Data';
    elements.cancelBtn.style.display = 'none';
}

/**
 * @param {Object} data
 */
function setEditMode(data) {
    isEditMode = true;
    elements.mahasiswaId.value = data.id;
    elements.nama.value = data.nama;
    elements.nim.value = data.nim;
    elements.jurusan.value = data.jurusan;
}

```



```

elements.email.value = data.email;

elements.formTitle.innerHTML = '<span
class="icon">✎</span> Edit Data Mahasiswa';
elements.submitBtn.innerHTML = '<span class="btn-
icon">📁</span> Update Data';
elements.cancelBtn.style.display = 'inline-flex';

elements.form.scrollToView({ behavior: 'smooth' });
}

/**
 * @param {string} title
 * @param {string} message
 * @param {function} onConfirm
 */
function showModal(title, message, onConfirm) {
  elements.modalTitle.textContent = title;
  elements.modalMessage.textContent = message;
  elements.modalOverlay.classList.add('active');

  elements.modalConfirm.onclick = () => {
    hideModal();
    onConfirm();
  };
}

function hideModal() {
  elements.modalOverlay.classList.remove('active');
}

async function fetchMahasiswa() {

```

```
toggleLoading(true);

try {
  const response = await fetch(API_URL);
  const result = await response.json();

  if (result.success) {
    renderTable(result.data);
  } else {
    showToast(result.message || 'Gagal mengambil data',
'error');
  }
} catch (error) {
  console.error('Fetch error:', error);
  showToast('Gagal terhubung ke server', 'error');
} finally {
  toggleLoading(false);
}
}

/**
 * @param {Object} data
 */
async function createMahasiswa(data) {
  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(data)
    });
  }
```

```

const result = await response.json();

if (result.success) {
  showToast('Data mahasiswa berhasil ditambahkan',
'success');
  resetForm();
  fetchMahasiswa();
} else {
  showToast(result.message || 'Gagal menambahkan data',
'error');
}
} catch (error) {
  console.error('Create error:', error);
  showToast('Gagal terhubung ke server', 'error');
}
}

/**
 * @param {number} id
 * @param {Object} data
 */
async function updateMahasiswa(id, data) {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(data)
    });
  }

```

```

const result = await response.json();

if (result.success) {
  showToast('Data mahasiswa berhasil diperbarui',
'success');
  resetForm();
  fetchMahasiswa();
} else {
  showToast(result.message || 'Gagal memperbarui data',
'error');
}
} catch (error) {
  console.error('Update error:', error);
  showToast('Gagal terhubung ke server', 'error');
}
}

/**
 * @param {number} id
 */
async function deleteMahasiswa(id) {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'DELETE'
    });

    const result = await response.json();

    if (result.success) {
      showToast('Data mahasiswa berhasil dihapus', 'success');
      fetchMahasiswa();
    } else {

```

```

        showToast(result.message || 'Gagal menghapus data',
        'error');
    }
    } catch (error) {
        console.error('Delete error:', error);
        showToast('Gagal terhubung ke server', 'error');
    }
}

/**
 * @param {Array} data
 */
function renderTable(data) {
    if (data.length === 0) {
        toggleEmptyState(true);
        return;
    }

    toggleEmptyState(false);

    elements.tableBody.innerHTML = data.map((mhs, index) => `
        <tr>
            <td>${index + 1}</td>
            <td>${escapeHtml(mhs.nama)}</td>
            <td>${escapeHtml(mhs.nim)}</td>
            <td>${escapeHtml(mhs.jurusan)}</td>
            <td>${escapeHtml(mhs.email)}</td>
            <td>
                <div class="action-buttons">
                    <button class="btn btn-success btn-sm"
onclick='handleEdit(${JSON.stringify(mhs)})>
                         Edit
                    </button>
                </div>
            </td>
        </tr>
    `);
}

```

```

        </button>

        <button class="btn btn-danger btn-sm"
onclick="handleDelete(${mhs.id},
'${escapeHtml(mhs.nama)}')">
             Hapus
        </button>
    </div>
</td>
</tr>
</tr>
    `).join(");
}

/**
 * @param {string} text
 */
function escapeHtml(text) {
    const div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

function handleSubmit(e) {
    e.preventDefault();

    const data = {
        nama: elements.nama.value.trim(),
        nim: elements.nim.value.trim(),
        jurusan: elements.jurusan.value.trim(),
        email: elements.email.value.trim()
    };

    if (!data.nama || !data.nim || !data.jurusan || !data.email) {

```

```

        showToast('Semua field harus diisi', 'warning');
        return;
    }

    if (isEditMode) {
        const id = elements.mahasiswaId.value;
        updateMahasiswa(id, data);
    } else {
        createMahasiswa(data);
    }
}

function handleEdit(data) {
    setEditMode(data);
}

/**
 * @param {number} id -
 * @param {string} nama
 */
function handleDelete(id, nama) {
    showModal(
        'Konfirmasi Hapus',
        `Apakah Anda yakin ingin menghapus data "${nama}"?`,
        () => deleteMahasiswa(id)
    );
}

elements.form.addEventListener('submit', handleSubmit);
elements.cancelBtn.addEventListener('click', resetForm);
elements.refreshBtn.addEventListener('click', fetchMahasiswa);
elements.modalCancel.addEventListener('click', hideModal);

```

```
elements.modalOverlay.addEventListener('click', (e) => {  
  if (e.target === elements.modalOverlay) {  
    hideModal();  
  }  
});  
  
document.addEventListener('DOMContentLoaded', () => {  
  fetchMahasiswa();  
});
```

c. Dokumentasi

Aplikasi ini merupakan sistem pengelolaan data mahasiswa berbasis web yang dibangun menggunakan teknologi Node.js sebagai runtime environment, Express.js sebagai framework backend, dan MySQL sebagai database. Aplikasi ini menyediakan RESTful API untuk operasi CRUD (Create, Read, Update, Delete) serta halaman web sederhana menggunakan HTML, CSS, dan JavaScript untuk mengakses API tersebut melalui browser.

i. Instalasi Proyek

Langkah pertama adalah membuat struktur proyek dan menginstall dependencies yang diperlukan. File package.json dibuat untuk mendefinisikan informasi proyek dan library yang digunakan.

Perintah yang dijalankan:

```
npm init -y
```

```
npm install express mysql
```

Dependencies yang digunakan:

- express: Framework web untuk membuat server HTTP dan routing

- mysql: Library untuk koneksi dan query ke database MySQL

ii. Konfigurasi Koneksi Database

File db.js dibuat untuk menangani koneksi ke database MySQL. Modul ini menggunakan library mysql untuk membuat koneksi dengan konfigurasi host localhost, user root, password kosong, dan database akademik. Koneksi ini kemudian di-export agar dapat digunakan oleh modul lain dalam aplikasi.

iii. Implementasi Operasi CRUD

File crud.js berisi fungsi-fungsi untuk melakukan operasi database. Setiap fungsi menggunakan callback pattern untuk menangani hasil query secara asynchronous.

Fungsi-fungsi yang dibuat:

- createMahasiswa(): Menjalankan query INSERT untuk menambah data mahasiswa baru ke database
- getAllMahasiswa(): Menjalankan query SELECT untuk mengambil seluruh data mahasiswa
- getMahasiswaById(): Menjalankan query SELECT dengan kondisi WHERE untuk mengambil data mahasiswa berdasarkan ID tertentu
- updateMahasiswa(): Menjalankan query UPDATE untuk memperbarui data mahasiswa yang sudah ada
- deleteMahasiswa(): Menjalankan query DELETE untuk menghapus data mahasiswa dari database

iv. Pembuatan Server Express dengan RESTful API

File app.js merupakan file utama yang menjalankan server Express. Server ini dikonfigurasi untuk menyajikan file statis dari folder public dan menyediakan endpoint API.

Endpoint RESTful API yang tersedia:

- GET /api/mahasiswa: Mengambil semua data mahasiswa
- GET /api/mahasiswa/:id: Mengambil data mahasiswa berdasarkan ID
- POST /api/mahasiswa: Menambahkan data mahasiswa baru
- PUT /api/mahasiswa/:id: Memperbarui data mahasiswa berdasarkan ID
- DELETE /api/mahasiswa/:id: Menghapus data mahasiswa berdasarkan ID

Setiap endpoint mengembalikan response dalam format JSON yang berisi status success, message, dan data.

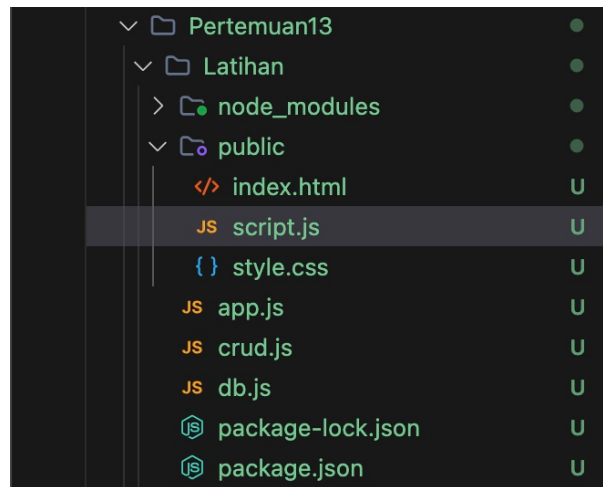
v. Pembuatan Halaman Web Frontend

Frontend aplikasi terdiri dari tiga file yang diletakkan dalam folder public:

- index.html: Halaman utama yang berisi struktur HTML meliputi form input untuk memasukkan data mahasiswa (nama, NIM, jurusan, email), tabel untuk menampilkan daftar mahasiswa, tombol aksi untuk edit dan hapus, modal konfirmasi untuk aksi hapus, dan container untuk toast notification.
- style.css: File stylesheet yang mengatur tampilan visual aplikasi dengan desain dark theme modern menggunakan efek glassmorphism, warna yang harmonis menggunakan HSL color system, animasi dan transisi untuk interaksi yang smooth, serta layout responsive yang menyesuaikan dengan berbagai ukuran layar.
- script.js: File JavaScript yang menangani logika frontend meliputi Fetch API untuk komunikasi dengan

server backend, fungsi-fungsi untuk menampilkan dan memanipulasi data di tabel, event handler untuk form submission dan tombol aksi, serta fungsi utility untuk toast notification dan modal konfirmasi.

vi. Struktur Folder Akhir



vii. Cara Menjalankan Aplikasi

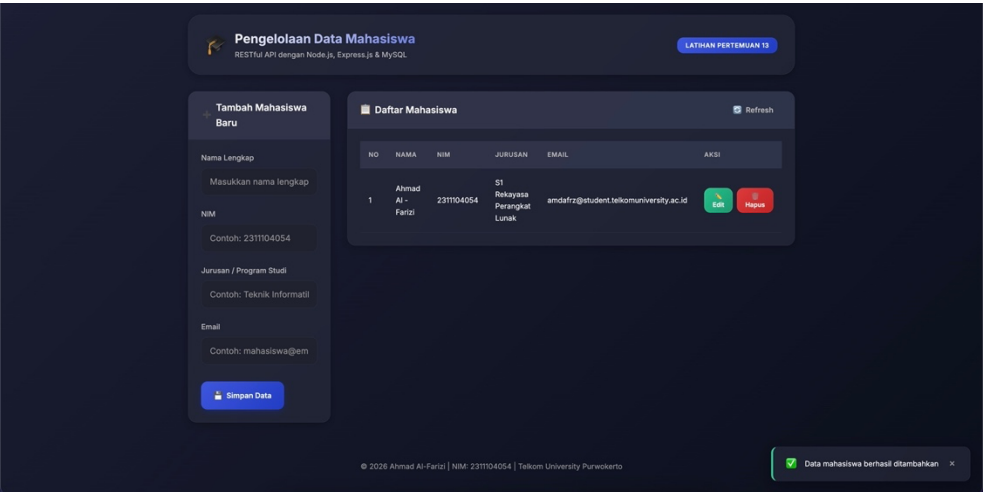
- Pastikan MySQL server sudah berjalan (XAMPP)
- Buat database dan tabel yang diperlukan
- Buka terminal dan navigasi ke folder Latihan
- Jalankan perintah `npm install` untuk menginstall dependencies
- Jalankan perintah `node app.js` untuk menjalankan server
- Buka browser dan akses `http://localhost:3000`

viii. Kesimpulan

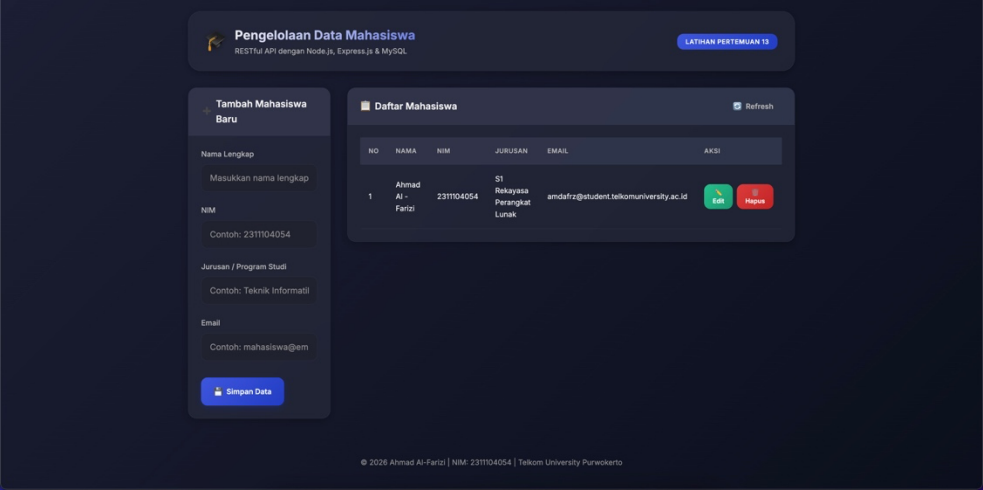
Aplikasi berhasil dibangun dengan mengimplementasikan arsitektur RESTful API di sisi backend dan halaman web interaktif di sisi frontend. Pengguna dapat melakukan operasi CRUD pada data mahasiswa melalui antarmuka web yang modern dan responsif. Komunikasi antara frontend dan backend dilakukan menggunakan Fetch API dengan format data JSON.

d. Screenshots

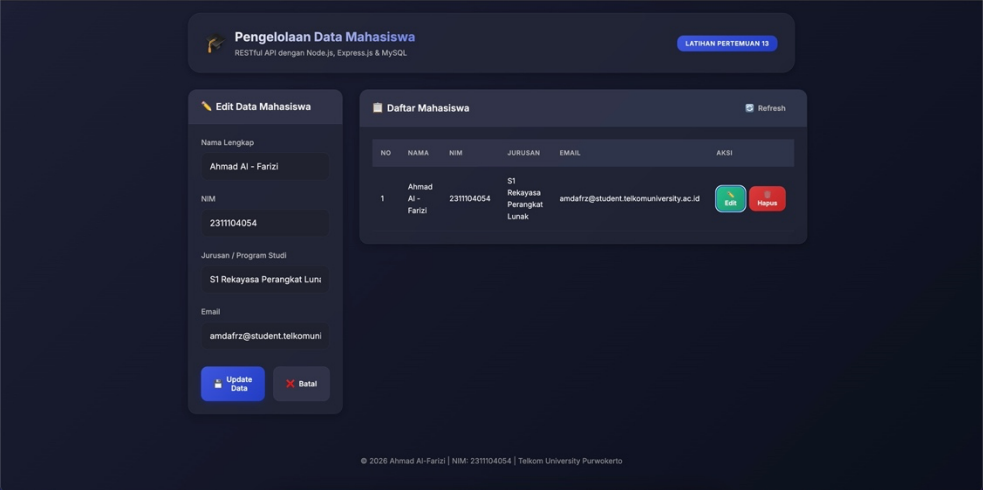
Create



Read



Update



Delete

**Pengelolaan Data Mahasiswa**
RESTful API dengan Node.js, Express.js & MySQL

LATIHAN PERTEMUAN 13

Tambah Mahasiswa Baru

Nama Lengkap

Masukkan nama lengkap

NIM

Contoh: 2311104054

Jurusan / Program Studi

Contoh: Teknik Informatika

Email

Contoh: mahasiswa@iem

Simpan Data

Daftar Mahasiswa

Refresh

NO	NAMA	NIM	JURUSAN	EMAIL	AKSI
1	Ahmad Al-Fariz	2311104054	S1 Rekayasa Perangkat Lunak	andafz@student.telkomuniversity.ac.id	Edit Hapus

© 2028 Ahmad Al-Fariz | NIM: 2311104054 | Telkom University Purwokerto

✓ Data mahasiswa berhasil ditambahkan

x

BAB III

PENUTUP

A. Kesimpulan

Praktikum Modul 13 membahas pembangunan RESTful API menggunakan Node.js, Express.js, dan MySQL. Node.js menyediakan lingkungan eksekusi JavaScript di sisi server dengan kemampuan asynchronous dan event-driven, sedangkan Express.js mempermudah pengelolaan routing dan middleware dalam aplikasi web. Melalui implementasi operasi CRUD berbasis RESTful API, mahasiswa dapat membangun sistem backend yang terstruktur, efisien, dan siap digunakan sebagai fondasi aplikasi web modern